```
>>>
         752013 function calls in 71.536 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.004    0.004   71.536   71.536 <string>:1(<module>)
        1    0.031    0.031    0.034    0.034 matrix_mult.py:12(zero)
        1   69.904   69.904   71.532   71.532 matrix_mult.py:15(mult)
        1    0.101    0.101    0.135    0.135 matrix_mult.py:6(__init__)
   250505    0.037    0.000    0.037    0.000 {len}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
   250000    0.032    0.000    0.032    0.000 {method 'random' of '_random.Random' objects}
   251503    1.427    0.000    1.427    0.000 {range}
```

Figure 1: cProfile in the CPython interpreter

```
    if len(matrix1[0]) != len(matrix2):
        print 'Matrices must be m*n and n*p to multiply!'
    else:
        new_matrix = matrix(len(matrix1), len(matrix2[0]))
        new_matrix.zero()
        for i in range(len(matrix1)):
            for j in range(len(matrix2[0])):
                for k in range(len(matrix2)):
                    new_matrix.mat[i][j] +=
matrix1[i][k]*matrix2[k][j]
    return new_matrix
```

The above code simply defines a class matrix, which generates a matrix of M rows and N columns, and initialises the matrix with random values. The function *zero* initialises the matrix to zero. The function *mult* takes two matrix objects, multiplies them and returns a matrix object with the result. The function *mult* takes *O(n^3)* time to multiply the matrices.

Profiling

Profiling is the process by which you analyse how a program is using system resources, and the time taken to execute portions of your code. By analysing your code, you can target potential 'hot spots', which are portions of your code that tend to run slowly. By optimising these areas of code, you will end up with a significantly faster execution time. In this article, let's focus on profiling your code with the following:

- cProfile
- runsnake
- line_profiler

cProfile

cProfile is the standard way to profile your Python code, by importing the *cProfile* module. To profile the above code using *cProfile*, add the following code:

```
import cProfile
def profile_mult(matrix1, matrix2):
    cProfile.run('mult(' + matrix1+'.mat,' + matrix2+'.mat)')

if __name__ == '__main__':
    x=matrix(500,500)
    y=matrix(500,500)
    profile_mult('x','y')
```

Figure 1 shows how you can use cProfile from the CPython interpreter. Note that the variables *x* and *y* should be passed as strings to the function *profile_mult*, because you need to pass a string that represents the statement to be profiled, to the *cProfile.run()* function. x and y are instances of the matrix class, each initialised with 500 rows and 500 columns. All values in the matrices are random values. Take a look at the matrix multiplication loop, *mult(matrix1, matrix2)*. It's iterating over *i*, *j*, and *k*—each of which is a column or row of a 500×500 matrix. 500^3 = 125 million cycles through Python for loops.

As you can see, cProfile gives you a lot of numbers, which represent the following:

1. *ncalls:* represents the number of calls made to the corresponding script or function.
2. *tottime:* represents the total time spent executing the script or function.
3. *percall*: represents the time taken per call to the function or script.
4. *cumtime:* is the time spent on the function, including calls to other functions.
5. *percall:* this is *cumtime* divided by *tottime*.

It also shows you the total number of function calls, and the amount of time taken to execute the statement.

Optimisations

Well, as we can see from the cProfile output, the functions *len*, *range*, and *random* are called the most number of times—due to which I can make the following optimisations in my *mult* function:

```
def mult(matrix1, matrix2):
    # Matrix multiplication
    if len(matrix1[0]) != len(matrix2):
      print 'Matrices must be m*n and n*p to multiply!'
```